4단계 주제 6/6

간단한 웹 화면 연결

코딩 · Azure AI 기초

K뉴딜 MS Copilot 활용 과정

Copilot부터 AI 에이전트 · 업무자동화

4단계 · 코딩 · Azure AI 기초

마치면 할 수 있어요

- [ ] Streamlit을 설치하고 내 컴퓨터에서 웹 앱을 실행할 수 있습니다.
- [ ] 채팅 입력창과 말풍선으로 RAG 함수를 연결할 수 있습니다.
- [ ] 답과 함께 근거를 펼쳐 볼 수 있는 화면을 만들 수 있습니다.
- [ ] 키가 드러나지 않게 결과물을 정리하고 실습 리소스를 삭제할 수 있습니다.

왜 Streamlit인가요

1. 터미널 챗봇은 만든 사람만 쓰기 좋습니다.
2. 면접관이나 친구에게 보여 주려면 화면이 필요합니다.
3. Streamlit은 HTML을 몰라도 Python만으로 웹 화면을 만드는 도구입니다.
4. 채팅 입력창과 말풍선 기능을 공식 문서에서 제공하고 튜토리얼도 있어 이 과정은 Streamlit 하나로 갑니다.
5. 지원하는 Python 버전은 설치 문서에서 확인해 보세요.

왜 Streamlit인가요

확인할 것	공식 문서
설치와 실행	docs.streamlit.io
채팅 앱 튜토리얼	docs.streamlit.io
채팅 입력창 기능	docs.streamlit.io

설치하고 시험 앱 띄우기 · Windows

1. 앞서 만든 가상환경을 켠 뒤 설치합니다.

설치하고 시험 앱 띄우기 · Windows

```
.venv\Scripts\Activate.ps1
pip install streamlit
streamlit hello
```

설치하고 시험 앱 띄우기 · macOS

1. 앞서 만든 가상환경을 켠 뒤 설치합니다.
2. Xcode 명령줄 도구가 필요하다는 안내가 나오면 먼저 설치해 보세요.

설치하고 시험 앱 띄우기 · macOS

```
source .venv/bin/activate
pip install streamlit
streamlit hello
```

설치하고 시험 앱 띄우기

1. **해 보기: 명령을 찾지 못한다면 앞에 python -m을 붙여 python -m streamlit hello로 실행해 보세요. 터미널이 이메일을 물으면 비워 두고 Enter를 눌러도 됩니다. 앱을 멈출 때는...**

Streamlit이 움직이는 방식과 화면 코드 (1/2)

1. 가장 먼저 알아 둘 특징이 있습니다.
2. Streamlit은 입력이 있을 때마다 파일을 위에서 아래로 다시 실행합니다.
3. 칠판에 쓴 글을 매번 지우고 다시 쓰는 셈입니다.
4. 그래서 대화 내용을 일반 변수에 두면 사라집니다.

Streamlit이 움직이는 방식과 화면 코드 (2/2)

1. 지워지지 않는 수첩인 st.session_state에 적어 둬야 합니다.
2. rag.py는 그대로 두고 app.py만 새로 만듭니다.
3. 화면과 두뇌를 나눠 둔 덕분에 화면을 바꿔도 RAG 코드를 건드리지 않습니다.

Streamlit이 움직이는 방식과 화면 코드

쓰는 기능	하는 일	비유
st.title, st.caption	제목과 안내 문구를 보여 줍니다	칠판 맨 윗줄
st.chat_input	아래쪽 채팅 입력창을 만듭니다	메시지 입력칸
st.chat_message	사용자와 챗봇 말풍선을 구분합니다	카카오톡 말풍선
st.session_state	다시 실행해도 남는 저장 공간입니다	지워지지 않는 수첩
st.spinner	기다리는 동안 움직이는 표시를 보여 줍니다	로딩 표시
st.expander	눌러야 펼쳐지는 상자를 만듭니다	접이식 메모

Streamlit이 움직이는 방식과 화면 코드 (1/2)

```python
import streamlit as st

from rag import answer

st.title("나만의 문서 챗봇")
st.caption("올려 둔 문서 안에서만 답합니다. 답 아래에서 근거를 확인해 보세요.")

if "messages" not in st.session_state:
    st.session_state.messages = []

def show_sources(chunks):
    """근거 조각을 접을 수 있는 상자에 보여 줍니다."""
    if chunks:
        with st.expander("근거 보기"):
            for number, chunk in enumerate(chunks, start=1):
                st.markdown(f"[{number}] {chunk['source']}")
```

> 주석을 포함한 전체 코드는 아래 본문에 있습니다.

Streamlit이 움직이는 방식과 화면 코드 (2/2)

```
                st.text(chunk["content"])

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])
        show_sources(message.get("chunks"))

if question := st.chat_input("문서 내용을 물어보세요"):
    st.session_state.messages.append({"role": "user", "content": question})
    with st.chat_message("user"):
        st.markdown(question)

    with st.chat_message("assistant"):
        try:
            with st.spinner("문서를 찾고 있어요"):
                text, chunks = answer(question)
```

> **주석을 포함한 전체 코드는 아래 본문에 있습니다.**

Streamlit이 움직이는 방식과 화면 코드

1. **Windows · macOS 같습니다.**
2. **터미널에 표시된 주소가 브라우저에서 열립니다.**

Streamlit이 움직이는 방식과 화면 코드

```
streamlit run app.py
```

키가 새지 않게 점검하기

1 **주의: 지금 만든 앱은 내 컴퓨터에서 보여 주는 프로토타입입니다. 로그인 없이 인터넷에 공개하면 누구나 내 구독 비용으로 모델을 부를 수 있으니 공개 배포하지 않습니다. 보여 줄 때는 캡처와 녹화를 쓰고 .env...**

키가 새지 않게 점검하기

1. Windows · macOS 같습니다.
2. 깃허브에 올리기 전 아래를 확인해 보세요.
3. .env가 목록에 보이면 멈추고 .gitignore부터 고칩니다.

키가 새지 않게 점검하기

```
git status

git check-ignore -v .env
```

> 주석을 포함한 전체 코드는 아래 본문에 있습니다.

프로토타입 정리하기

결과물에 넣을 것	내용
README	무엇을 하는 챗봇인지, 어떤 문서를 넣었는지, 실행 방법
구조도	질문이 색인과 모델을 거쳐 답이 되는 흐름
화면 캡처	근거를 펼친 답변 화면, 문서에 없는 질문에 답하는 화면
배운 점	틀렸던 답과 고친 방법, 키워드 검색의 한계
requirements.txt	pip freeze > requirements.txt 로 만듭니다. 다른 사람이 같은 환경을 설치할 수 있습니다

프로토타입 정리하기

1. 마지막으로 비용을 끊습니다.
2. 결과를 캡처하고 코드를 저장했다면 Azure 포털 · 리소스에서 익힌 순서대로 배포를 지우고 rg-rag-study를 삭제해 보세요.
3. 코드와 문서가 내 컴퓨터에 있으니 다시 필요할 때 리소스를 새로 만들어 .env만 바꾸면 됩니다.

4단계 · 코딩 · Azure AI 기초

실습

시연을 보고, 같은 결과를 따라 만든 뒤, 내 주제로 바꿉니다.

실습 1

시연을 봅니다

실습

1 터미널 챗봇에 말풍선 화면을 입히고 답 아래에서 근거를 펼쳐 원문과 대조하는 모습을 함께 봅니다.

실습 2

따라 합니다 (1/2)

실습

1. 가상환경을 켜고 Streamlit을 설치합니다.
2. streamlit hello로 시험 앱이 뜨는지 확인합니다.
3. app.py를 프로젝트 폴더에 저장합니다.
4. streamlit run app.py를 실행합니다.

실습 2

따라 합니다 (2/2)

실습

5 문서에 있는 질문을 입력합니다.

6 근거 보기를 펼쳐 원문과 대조합니다.

7 제목과 안내 문구를 내 챗봇에 맞게 바꿉니다.

8 캡처한 뒤 배포와 리소스 그룹을 삭제합니다.

실습 2

복사해서 쓰는 프롬프트

```
내 프로젝트의 app.py를 읽고 Streamlit이 입력마다 파일을 다시 실행한다는 점을 줄을 짚어 설명
    해줘. 그다음 화면에 [추가할 기능, 예: 대화 지우기 버튼]을 넣어줘. rag.py는 고치지 말고
    키도 코드에 쓰지 말아줘. 바꾼 줄마다 한국어 주석을 달아줘.
```

실습 2

체크포인트

시점	이렇게 되면 통과
10분	streamlit hello 화면이 브라우저에 뜹니다.
20분	내 앱의 제목과 입력창이 보입니다.
30분	질문하면 말풍선에 답이 나오고 이전 대화도 남아 있습니다.
40분	근거 보기를 펼치면 파일 이름과 조각 본문이 보입니다.
50분	git status에 .env가 없고 포털에서 rg-rag-study가 지워졌습니다.

실습 3

내 주제로 바꿉니다

실습

1. 제목과 안내 문구, 첫 인사말을 내 주제에 맞게 바꿔 보세요.
2. 자주 묻는 질문 버튼을 더하면 처음 보는 사람도 바로 시험해 볼 수 있습니다.

과제

기본 과제와 도전 과제

기본	내 문서로 답하는 채팅 화면을 완성해 근거를 펼친 화면을 캡처해 보세요. 캡처에 키가 없는지 확인합니다.
도전	대화 지우기 버튼이나 답변 평가 버튼을 Copilot과 함께 넣어 보세요. 이전 대화를 기억해 이어 묻는 기능도 어떻게 만들지 설계해 적어 보세요.

코딩 · Azure AI 기초 마무리

간단한 웹 화면 연결

포트폴리오에 넣을 것: 내 문서로 근거를 달아 답하는 RAG 챗봇 프로토타입(코드, README, 화면 캡처)

- ✓ **Streamlit을 설치하고 내 컴퓨터에서 웹 앱을 실행할 수 있습니다.**
- ✓ **채팅 입력창과 말풍선으로 RAG 함수를 연결할 수 있습니다.**
- ✓ **답과 함께 근거를 펼쳐 볼 수 있는 화면을 만들 수 있습니다.**
- ✓ **키가 드러나지 않게 결과물을 정리하고 실습 리소스를 삭제할 수 있습니다.**